

fxml之三



# 类型的实例化

---

北京理工大学计算机学院  
金旭亮

# 概述



JavaFX的视图文件中，以声明方式定义了JavaFX应用的可视化界面，在运行时，会被实例化为一棵“控件树”，传给Scene之后，变成“场景图”。



JavaFX控件包容诸多的属性，这些属性的类型多种多样，因此，fxml必须支持以声明的方式实例化多种类型的对象作为这些属性的值。



本讲通过示例，介绍如何在fxml文档中实例化各种类型的对象，掌握这些内容，对于编写fxml文档来说是必需的，因为你往往需要在fxml文档中以声明的方式初始化特定控件特定属性的值。

# 调用默认构造方法实例化大多数类型



在fxml中，以大写字母开头的元素，通常都对应着相应的JavaFX控件类型：

```
<VBox ...>
```

```
</VBox>
```



JavaFX在运行时，会调用这些类型的“默认构造方法”创建实例，再使用反射来给相应的属性赋值。由于用到了反射，所以，JavaFX项目的module-info.java中，需要使用open语句给特定的包开放“反射”权限。

# 通过valueOf()方法实例化

没有定义默认构造方法，但提供了valueOf()方法的类型，可以使用fx:value来实例化，最常见的就是用此方式来实例化包装类型：

```
<String fx:value="Hello, World!"/>  
<Double fx:value="1.0"/>  
<Boolean fx:value="false"/>
```

# BuilderFactory

JavaFX有一个内置的JavaFXBuilderFactory，可用来创建特定的类型的实例，常见的用法是实例化那些只读类型，比如Color：

```
<Color red="1.0" green="0.0" blue="0.0"/>
```



等价于

```
<Color>  
  <red>1.0</red>  
  <green>0.0</green>  
  <blue>0.0</blue>  
</Color>
```

这个实例化过程，对于开发者来说其实是“透明的”，无需理会。

# 应用实例

在fxml文档中实例化一个Label对象，并设定其特定属性值：

```
<Label>
    <font>
        <Font size="20"/>
    </font>
    <textFill>
        <!--使用Builder实例化Color对象-->
        <Color red="1.0" green="0.0" blue="0.0"/>
    </textFill>
    <!-- 调用valueOf()方法-->
    <Double fx:value="1.0"/>
</Label>
```



1.0

# 自定义类型的实例化

```
public class MyTestClass {  
  
    1 usage  
    public static final int MAX_VALUE = 255;  
    3 usages  
    private int num;  
    3 usages  
    private String info;  
  
>    getter and setter  
  
    @Override  
>    public String toString() {...}  
}
```

自定义类型中的字段，应该提供getter或setter方法，也可以定义常量。

```
<?import com.jinxuliang.learnfxml.MyTestClass?>
```

```
<Label>  
    <!--实例化自定义类型-->  
    <MyTestClass>  
        <info>  
            <!-- 调用valueOf()方法实例化对象-->  
            <String fx:value="info"/>  
        </info>  
        <num>  
            <Integer fx:value="100"/>  
        </num>  
    </MyTestClass>  
</Label>  
<Label>  
    <!--引用常量-->  
    <MyTestClass fx:constant="MAX_VALUE"/>  
</Label>
```

MyClass{num=100, info='info'}

255

# 使用对象工厂实例化类



```
public class MySingleton {  
    1 usage  
    private static MySingleton INSTANCE = new MySingleton();  
  
    1 usage  
    private MySingleton() {  
    }  
  
    2 usages  
    public static MySingleton getInstance() {  
        return INSTANCE;  
    }  
}
```

工厂方法不能有参数。

```
<!--使用对象工厂实例化Singleton对象，会拥有相同的hashCode值-->  
<Label>  
    <MySingleton fx:factory="getInstance"/>  
</Label>  
<Label>  
    <MySingleton fx:factory="getInstance"/>  
</Label>
```



com.jinxuliang.classinstance.model.MySingleton@214545dd

com.jinxuliang.classinstance.model.MySingleton@214545dd



# 集合对象的实例化

```
<Label>  
  <!-- 实例化Map-->  
  <HashMap foo="123" bar="456"/>  
</Label>
```

{bar=456, foo=123}

```
<ListView>  
  <items>  
    <!-- 使用工厂方法实例化List-->  
    <FXCollections fx:factory="observableArrayList">  
      <String fx:value="第一项"/>  
      <String fx:value="第二项"/>  
      <String fx:value="第三项"/>  
    </FXCollections>  
  </items>  
</ListView>
```

第一项  
第二项  
第三项

运行截图

# 定义可重用的对象

`<!--定义可重用的对象-->`

```
<fx:define>
```

```
    <Insets fx:id="padding" bottom="5.0" left="5.0" right="5.0" top="5.0"/>
```

```
    <Image url="@code.jpg" fx:id="myImg"/>
```

```
    <ToggleGroup fx:id="myToggleGroup"/>
```

```
</fx:define>
```

使用fx:define定义可重用的对象，使用fx:reference或“\$”表达式使用它们。

`<!--使用可重用的对象-->`

```
<Label padding="$padding">
```

重用Padding值的Label

```
</Label>
```

```
<ImageView>
```

```
    <fx:reference source="myImg"/>
```

```
</ImageView>
```

```
<RadioButton text="A" toggleGroup="$myToggleGroup"/>
```

```
<RadioButton text="B" toggleGroup="$myToggleGroup"/>
```

```
<RadioButton text="C" toggleGroup="$myToggleGroup"/>
```



重用Padding值的Label



☐ A

☐ B

☐ C